UNIT -I

INTRODUCTION OF COMPILER DESIGN
LECTURE NO: 1
DEFINITION OF COMPILER, INTERPRETER, TRANSLATOR

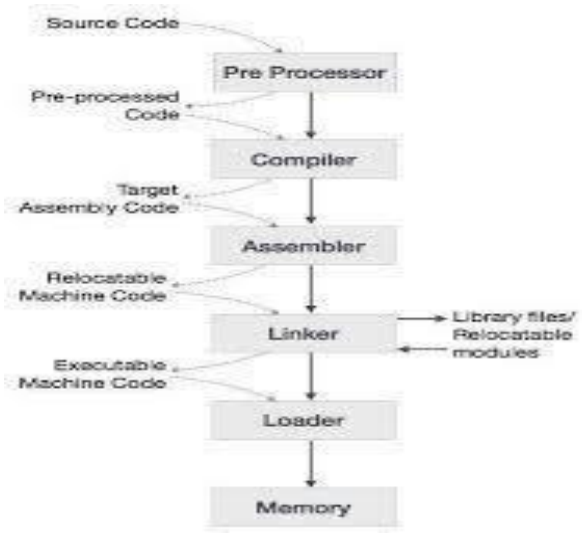


FIG: 1.1 PREPROCESSOR

PREPROCESSOR

A preprocessor produce input to compilers. They may perform the following functions.

1. **Macro processing:** A preprocessor may allow a user to define macros that are short hands for longer constructs.
2. **File inclusion:** A preprocessor may include header files into the program text.
3. **Rational preprocessor:** these preprocessors augment older languages with more modern flow-of-control and data structuring facilities.
4. **Language Extensions:** These preprocessor attempts to add capabilities to the language by certain amounts to build-in macro

COMPILER

Compiler is a translator program that translates a program written in (HLL) the source. program and translates it into an equivalent program in (MLL) the target program.

Executing a program written n HLL programming language is basically of two parts. the source program must first be compiled translated into a object program. Then the results object program is loaded into a memory executed.

ASSEMBLER

Programmers found it difficult to write or read programs in machine language. They begin to use a mnemonic (symbols) for each machine instruction, which they would subsequently translate into machine language.

Such a mnemonic machine language is now called an assembly language. Programs known as assembler were written to automate the translation of assembly language into machine language. The input to an assembler program is called source program, the output is a machine language translation (object program).

INTERPRETER

An interpreter is a program that appears to execute a source program as if it were machine language.

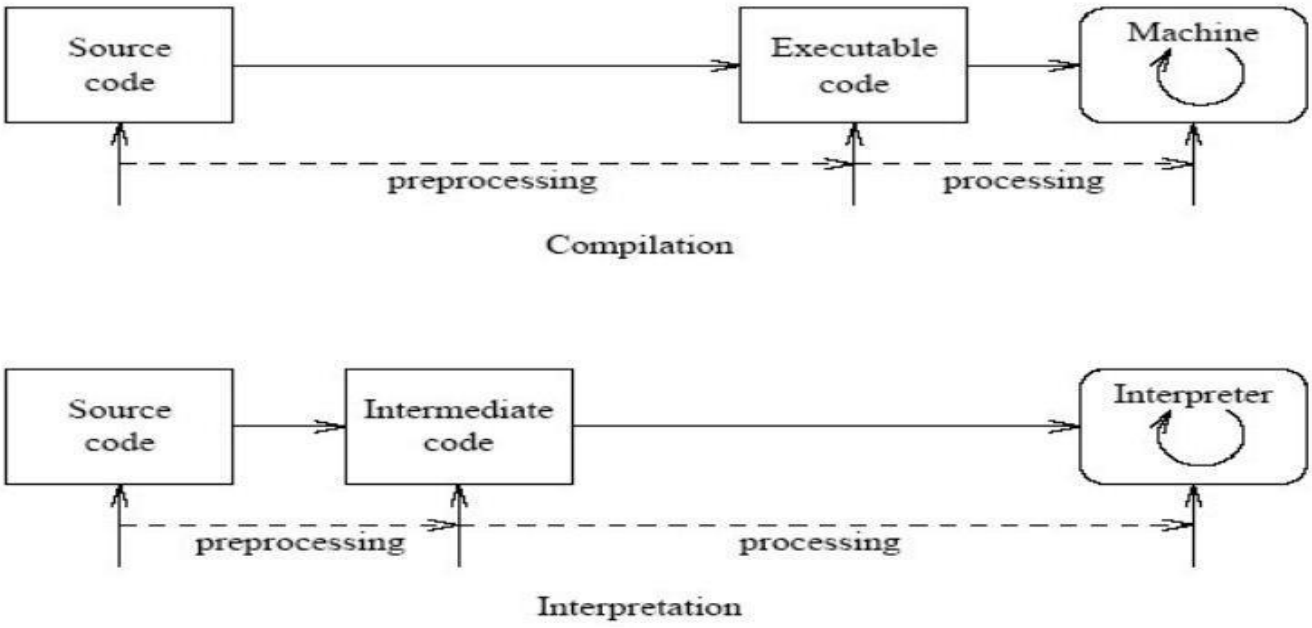


FIG: 1.2
INTERPRETER

Languages such as BASIC, SNOBOL, LISP can be translated using interpreters. JAVA also uses interpreter. The process of interpretation can be carried out in following phases.

1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Direct Execution

Advantages:

1. Modification of user program can be easily made and implemented as execution proceeds.
2. Type of object that denotes various may change dynamically.
3. Debugging a program and finding errors is simplified task for a program used for interpretation.
4. The interpreter for the language makes it machine independent.

Disadvantage

1. Speed of execution becomes slower.
2. Memory consumption is more.

LOADER AND LINK-EDITOR:

Once the assembler procedures an object program, that program must be p laced into memory and executed. The assembler could place the object program directly in memory and transfer control to it, thereby causing the machine language program to be execute.

This would waste core by leaving the assembler in memory while the user's program was being executed. Also, the programmer would have to retranslate his program with each execution, thus wasting translation time. To overcome this problems of wasted translation time and memory. System programmers developed another component called loader.

"A loader is a program that places programs into memory and prepares them for execution." It would be more efficient if subroutines could be translated into object form the loader could" relocate" directly behind the user's program.

TRANSLATOR

A translator is a program that takes as input a program written in one language and produces as output a program in another language. Beside program translation, the translator performs another very important role, the error-detection. Any violation of d HLL specification would be detected and reported to the programmers. Important role of translator is:

1. Translating the high-level language program input into an equivalent ml program.

2.Providing diagnostic messages wherever the programmer violates specification.

TYPES OF TRANSLATOR

1. COMPILER
2. INTERPRETER

TYPES OF COMPILER

1. BASIC compilers
2. C# compilers
3. C++ compilers
4. COBOL compilers
5. Java compiler

PHASES OF COMPILER

PHASES OF A COMPILER:

A compiler operates in phases. A phase is a logically interrelated operation that takes source program in one representation and produces output in another representation. The phases of a compiler are shown in below

There are two phases of compilation.

a. Analysis (Machine Independent/Language Dependent)

Synthesis (Machine Dependent/Language independent) Compilation process is partitioned into no-of-sub processes called **'phases.**

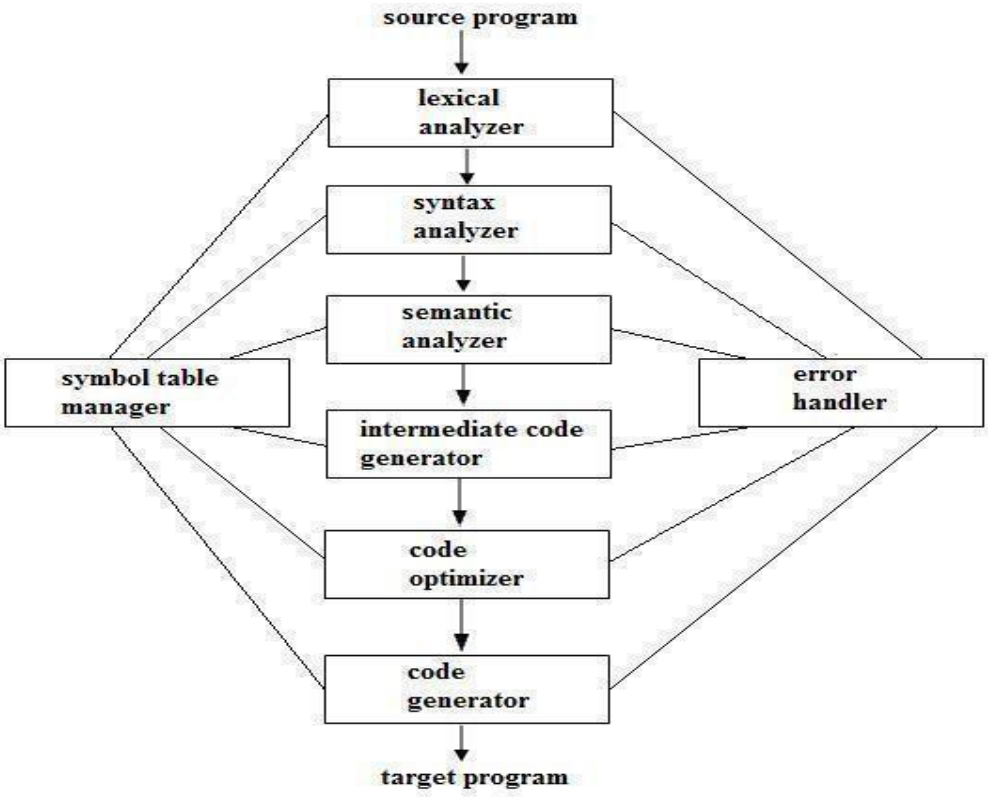


Fig 1.5 Phases of a compiler

Lexical Analysis: -

LA or Scanners reads the source program one character at a time, carving the source program into a sequence of automatic units called **tokens.**

Syntax Analysis: -

The second stage of translation is called syntax analysis or parsing. In this phase expressions, statements, declarations etc… are identified by using the results of lexical analysis. Syntax analysis is aided by using techniques based on formal grammar of the programming language.

Intermediate Code Generations: -

An intermediate representation of the final machine language code is produced. This phase bridges the analysis and synthesis phases of translation.

Code Optimization: -

This is optional phase described to improve the intermediate code so that the output runs faster and takes less space.

Code Generation: -

The last phase of translation is code generation. A number of optimizations to **Reduce the length of machine language program** are carried out during this phase. The output of the code generator is the machine language program of the specified computer.

Table Management (or) Book-keeping: -

This is the portion to **keep the names** used by the program and records essential information about each. The data structure used to record this information called a 'Symbol Table'.

LECTURE NO-2
BOOTSTRAPPING

Bootstrapping:

When a computer is first turned on or restarted, a special type of absolute loader, called as bootstrap loader is executed. This bootstrap loads the first program to be run by the computer usually an operating system. The bootstrap itself begins at address O in the memory of the machine. It loads the operating system (or some other program) starting at address 80. After all of the object code from device has been loaded, the bootstrap program jumps to address 80, which begins the execution of the program that was loaded.

Such loaders can be used to run stand-alone programs independent of the operating system or the system loader. They can also be used to load the operating system or the loader itself into memory.

Loaders are of two types:

- Linking loader.
- Linkage editor.

Linkage loaders perform all linking and relocation at load time.

Linkage editors perform linking prior to load time and dynamic linking, in which the linking function is performed at execution time.

A linkage editor performs linking and some relocation; however, the linkage program is written to a file or library instead of being immediately loaded into memory. This approach reduces the overhead when the program is executed. All that is required at load time is a very simple form of relocation.

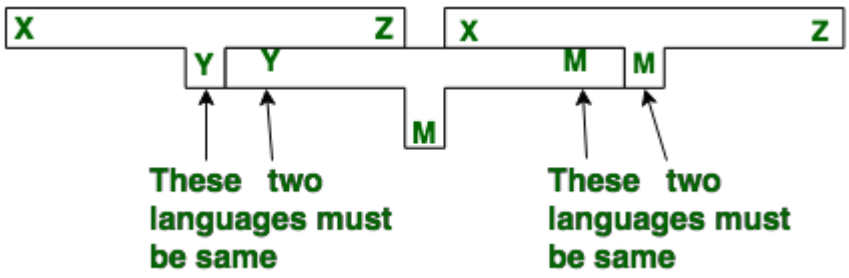
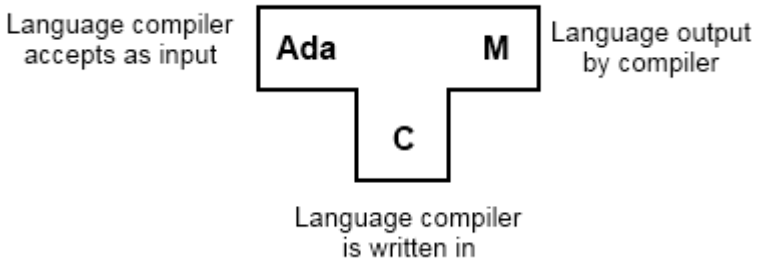


FIG: 1.4 BOOTSTRAPPING


Language compiler accepts as input
Ada
M
Language output by compiler
C
Language compiler is written in

LECTURE NO-3

REVIEW OF FINITE AUTOMATA LEXICAL ANLYZER

Finite automata is a state machine that takes a string of symbols as input and changes its state accordingly. Finite automata is a recognizer for regular expressions. When a regular expression string is fed into finite automata, it changes its state for each literal. If the input string is successfully processed and the automata reaches its final state, it is accepted, i.e., the string just fed was said to be a valid token of the language in hand.

The mathematical model of finite automata consists of:

- Finite set of states (Q)
- Finite set of input symbols (Σ)
- One Start state (q0)
- Set of final states (qf)
- Transition function (δ)

The transition function (δ) maps the finite set of state (Q) to a finite set of input symbols (Σ), $Q \times \Sigma \rightarrow Q$

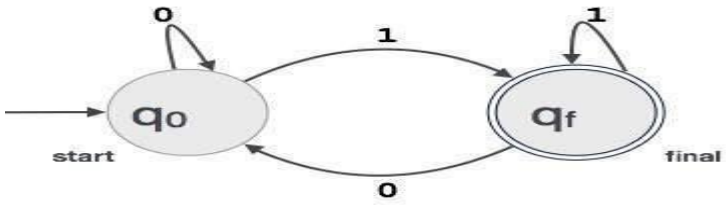
Finite Automata Construction

Let L(r) be a regular language recognized by some finite automata (FA).

- **States**: States of FA are represented by circles. State names are written inside circles.
- **Start state**: The state from where the automata starts, is known as the start state. Start state has an arrow pointed towards it.
- **Intermediate states**: All intermediate states have at least two arrows; one pointing to and another pointing out from them.
- **Final state**: If the input string is successfully parsed, the automata is expected to be in this state. Final state is represented by double circles. It may have any odd number of arrows pointing to it and even number of arrows pointing out from it. The number of odd arrows is one greater than even, i.e. **odd = even+1**.

- **Transition**: The transition from one state to another state happens when a desired symbol in the input is found. Upon transition, automata can either move to the next state or stay in the same state. Movement from one state to another is shown as a directed arrow, where the arrows points to the destination state. If automata stay on the same state, an arrow pointing from a state to itself is drawn.

Example: We assume FA accepts any three digit binary value ending in digit 1. FA = {Q(q_0, q_f), Σ(0,1), q_0, q_f, δ}



LEX:

- Lex is a program that generates lexical analyzer. It is used with YACC parser generator.
- The lexical analyzer is a program that transforms an input stream into a sequence of tokens.
- reads the input stream and produces the source code as output through implementing the lexical analyzer in the C program.

The function of Lex is as follows:

- Firstly lexical analyzer creates a program lex.1 in the Lex language. Then Lex compiler runs the lex.1 program and produces a C program lex.yy.c.
- Finally C compiler runs the lex.yy.c program and produces an object program a.out.
- a.out is lexical analyzer that transforms an input stream into a sequence of tokens.

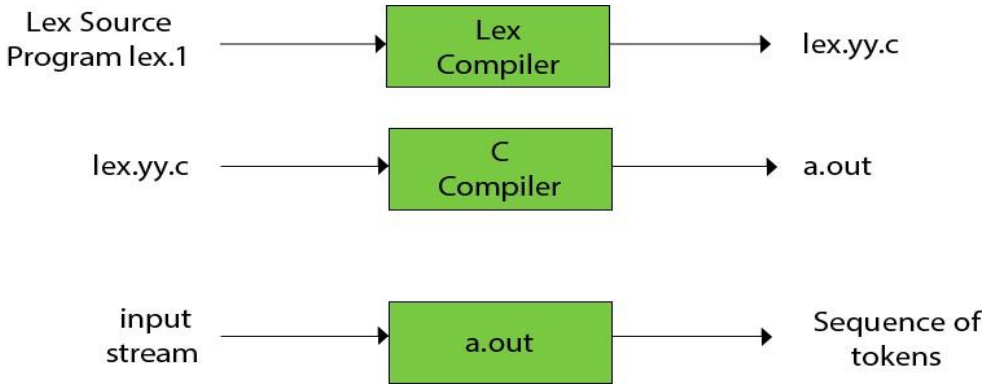


FIG: 1.5 LEX

LECTURE 4
INPUT BUFFERING

INPUT BUFFERING

To ensure that a right lexeme is found, one or more characters have to be looked up beyond the next lexeme.

• Hence a two-buffer scheme is introduced to handle large lookaheads safely.

• Techniques for speeding up the process of lexical analyzer such as the use of sentinels to mark the buffer end have been adopted.

There are three general approaches for the implementation of a lexical analyzer:

(i) By using a lexical-analyzer generator, such as lex compiler to produce the lexical analyzer from a regular expression-based specification. In this, the generator provides routines for reading and buffering the input.

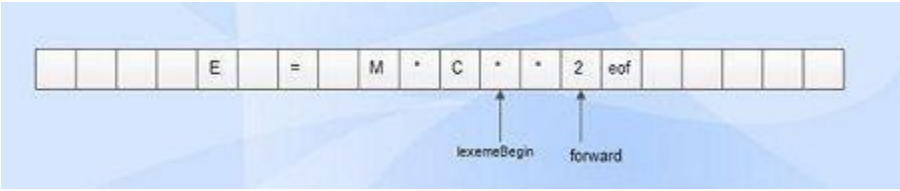
(ii) By writing the lexical analyzer in a conventional systems-programming language, using I/O facilities of that language to read the input.

(iii) By writing the lexical analyzer in assembly language and explicitly managing the reading of input.

Buffer Pairs

Because of large amount of time consumption in moving characters, specialized buffering techniques have been developed to reduce the amount of overhead required to process an input character.

Fig shows the buffer pairs which are used to hold the input data.



Scheme

Consists of two buffers, each consists of N-character size which are reloaded alternatively.

• N-Number of characters on one disk block, e.g., 4096.

• N characters are read from the input file to the buffer using one system read command.

• *eof* is inserted at the end if the number of characters is less than N.

Pointers

Two pointers lexemeBegin and forward are maintained.

lexeme Begin points to the beginning of the current lexeme which is yet to be found.

forward scans ahead until a match for a pattern is found.

• Once a lexeme is found, lexemebegin is set to the character immediately after the lexeme which is just found and forward is set to the character at its right end.

• Current lexeme is the set of characters between two pointers.

Disadvantages of this scheme

• This scheme works well most of the time, but the amount of lookahead is limited.

• This limited lookahead may make it impossible to recognize tokens in situations where the distance that the forward pointer must travel is more than the length of the buffer.

(eg.) DECLARE (ARGl, ARG2, . . . , ARGn) in PL/1 program;

• It cannot determine whether the DECLARE is a keyword or an array name until the character that follows the right parenthesis.

Sentinels

• In the previous scheme, each time when the forward pointer is moved, a check is done to ensure that one half of the buffer has not moved off. If it is done, then the other half must be reloaded.

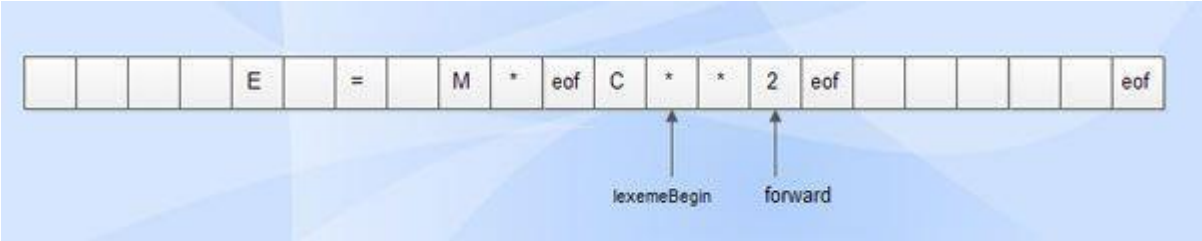
• Therefore, the ends of the buffer halves require two tests for each advance of the forward pointer.

Test 1: For end of buffer.

Test 2: To determine what character is read.

• The usage of sentinel reduces the two tests to one by extending each buffer half to hold a sentinel character at the end.

• The sentinel is a special character that cannot be part of the source program. *(eof* character is used as sentinel).



Advantages

• Most of the time, It performs only one test to see whether forward pointer points to an *eof.*

• Only when it reaches the end of the buffer half or *eof,* it performs more tests.

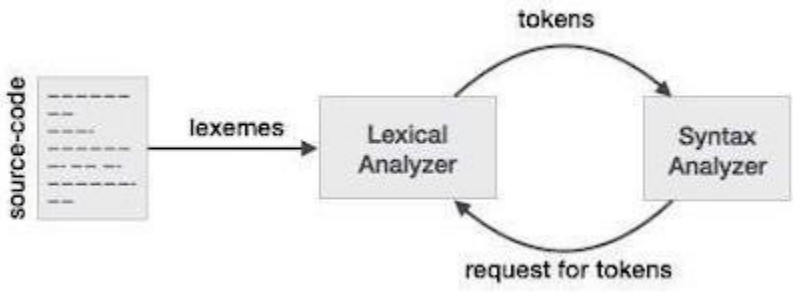
• Since N input characters are encountered between *eofs,* the average number of tests per input character is very close to 1.

RECOGNITION OF TOKENS

First know about Lexical Analysis:

1. The lexical analyzer breaks syntaxes into a series of **tokens**, by removing any whitespace or comments in the source code.

2. If the lexical analyzer finds a token invalid, it generates an error. It reads character streams from the source code, checks for legal tokens, and passes the data to the syntax analyzer when it demands.



What is Token?

In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

For example, in C language, the variable declaration line
int value = 100;

contains the tokens:

int (keyword),
value (identifier),
= (operator),
100 (constant) and
; (symbol)

Lexeme	Token
=	EQUAL_OP
*	MULT_OP
,	COMMA
(	LEFT_PAREN

Specifications of Tokens:

Let us understand how the language theory undertakes the following terms:

1. Alphabets
2. Strings
3. Special symbols
4. Language
5. Longest match rule
6. Operations
7. Notations
8. Representing valid tokens of a language in regular expression
9. Finite automata

1. Alphabets:

Any finite set of symbols

- {0,1} is a set of binary alphabets,
- {0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F} is a set of Hexadecimal alphabets,
- {a-z, A-Z} is a set of English language alphabets.

2. Strings: Any finite sequence of alphabets is called a string.

3. Special symbols: A typical high-level language contains the following symbols:

Arithmetic Symbols	Addition(+), Subtraction(-), Multiplication(*), Division(/)
Punctuation	Comma(,), Semicolon(;), Dot(.)
Assignment	=
Special assignment	+=, -=, *=, /=
Comparison	==, !=. <. <=. >, >=
Preprocessor	#

4. Language: A language is considered as a finite set of strings over some finite set of alphabets.

5. Longest match rule: When the lexical analyzer read the source-code, it scans the code letter by letter and when it encounters a whitespace, operator symbol, or special symbols it decides that a word is completed.

Operations: The various operations on languages are:

Union of two languages L and M is written as, L U M = {s | s is in L or s is in M}
Concatenation of two languages L and M is written as, LM = {st | s is in L and t is in M}
The Kleene Closure of a language L is written as, L* = Zero or more occurrence of language L.

7. Notations: If r and s are regular expressions denoting the languages L(r) and L(s), then

1. **Union** : L(r)UL(s)
2. **Concatenation** : L(r)L(s)
3. **Kleene closure** : (L(r))*

8. Representing valid tokens of a language in regular expression:

If x is a regular expression, then:

- x* means zero or more occurrence of x.
- x+ means one or more occurrence of x.

9. Finite automata: Finite automata is a state machine that takes a string of symbols as input and changes its state accordingly.

If the input string is successfully processed and the automata reaches its final state, it is accepted.

The mathematical model of finite automata consists of:

- finite set of states (Q)
- Finite set of input symbols (Σ)
- One Start state (q0)
- Set of final states (qf)
- Transition function (δ)

The transition function (δ) maps the finite set of state (Q) to a finite set of input symbols (Σ), $Q \times \Sigma \rightarrow Q$

LECTURE 5:
IDEA ABOUT LEX: A LEXICAL ANALYZER

LEX INTRODUCTION:

Lex is **a program that generates lexical analyzer**. It is used with YACC parser generator. The lexical analyzer is a program that transforms an input stream into a sequence of tokens.

It reads the input stream and produces the source code as output through implementing the lexical analyzer in the C program.

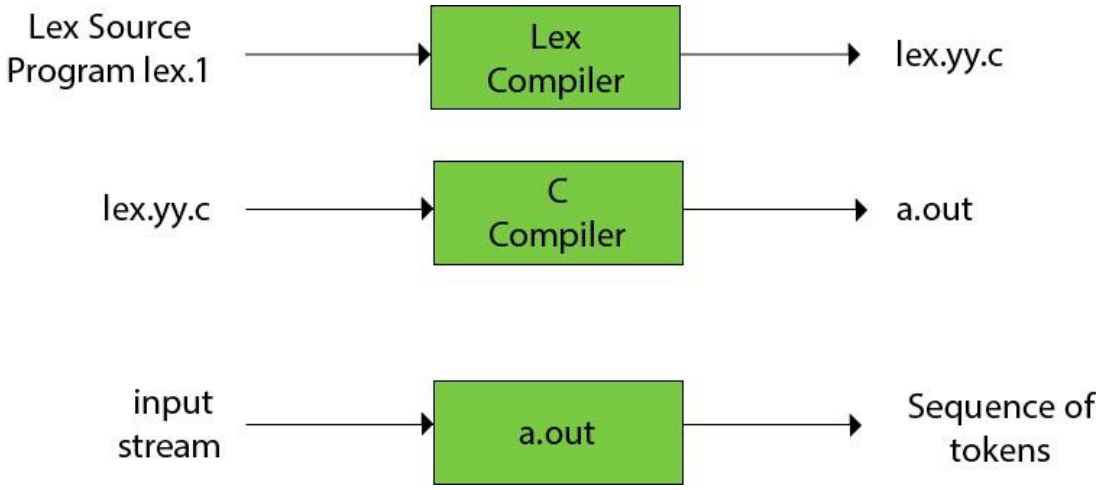


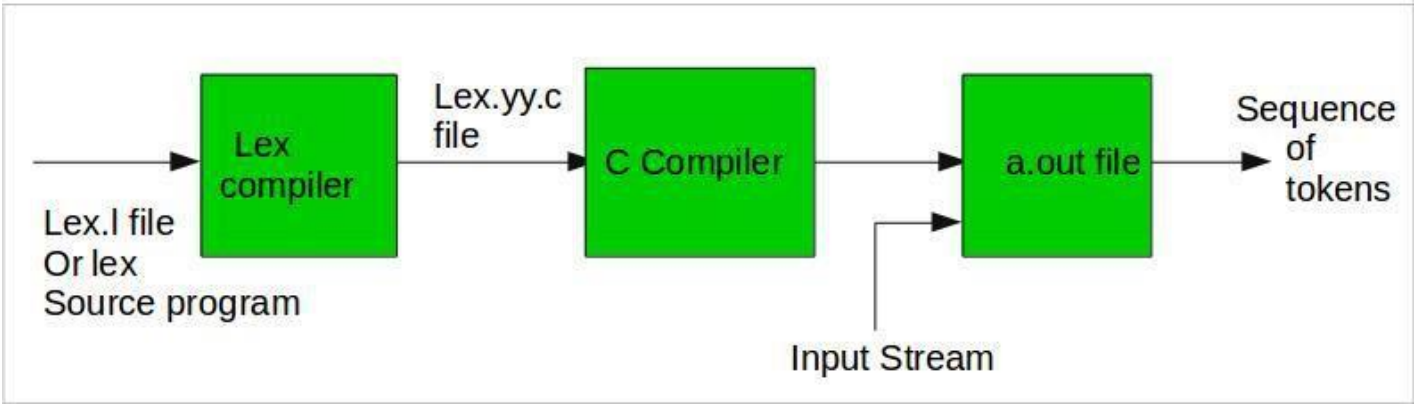
FIG: 1.7 LEXICAL ANALYZER

Lexical Analysis is the very first phase in the compiler designing. A Laxer takes the modified source code which is written in the form of sentences. In other words, it helps you to convert a sequence of characters into a sequence of tokens.

The lexical analyzer breaks this syntax into a series of tokens. It removes any extra space or comment written in the source code.

Programs that perform Lexical Analysis in compiler design are called lexical analyzers or lexers. A lexer contains tokenizer or scanner. If the lexical analyzer detects that the token is invalid, it generates an error.

The role of Lexical Analyzer in compiler design is to read character streams from the source code, check for legal tokens, and pass the data to the syntax analyzer when it demands.



Example

How Pleasant Is the Weather?

See this Lexical Analysis example; Here, we can easily recognize that there are five words How Pleasant, The, Weather, Is. This is very natural for us as we can recognize the separators, blanks, and the punctuation symbol.

HowPl easantIs Th ewe ather?

Now, check this example, we can also read this. However, it will take some time because separators are put in the Odd Places. It is not something which comes to you immediately.

Advantages of Lexical analysis

- Lexical analyzer method is used by programs like compilers which can use the parsed data from a programmer's code to create a compiled binary executable code

  It is used by web browsers to format and display a web page with the help of parsed data from JavsScript, HTML, CSS

- A separate lexical analyzer helps you to construct a specialized and potentially more efficient processor for the task

Disadvantage of Lexical analysis

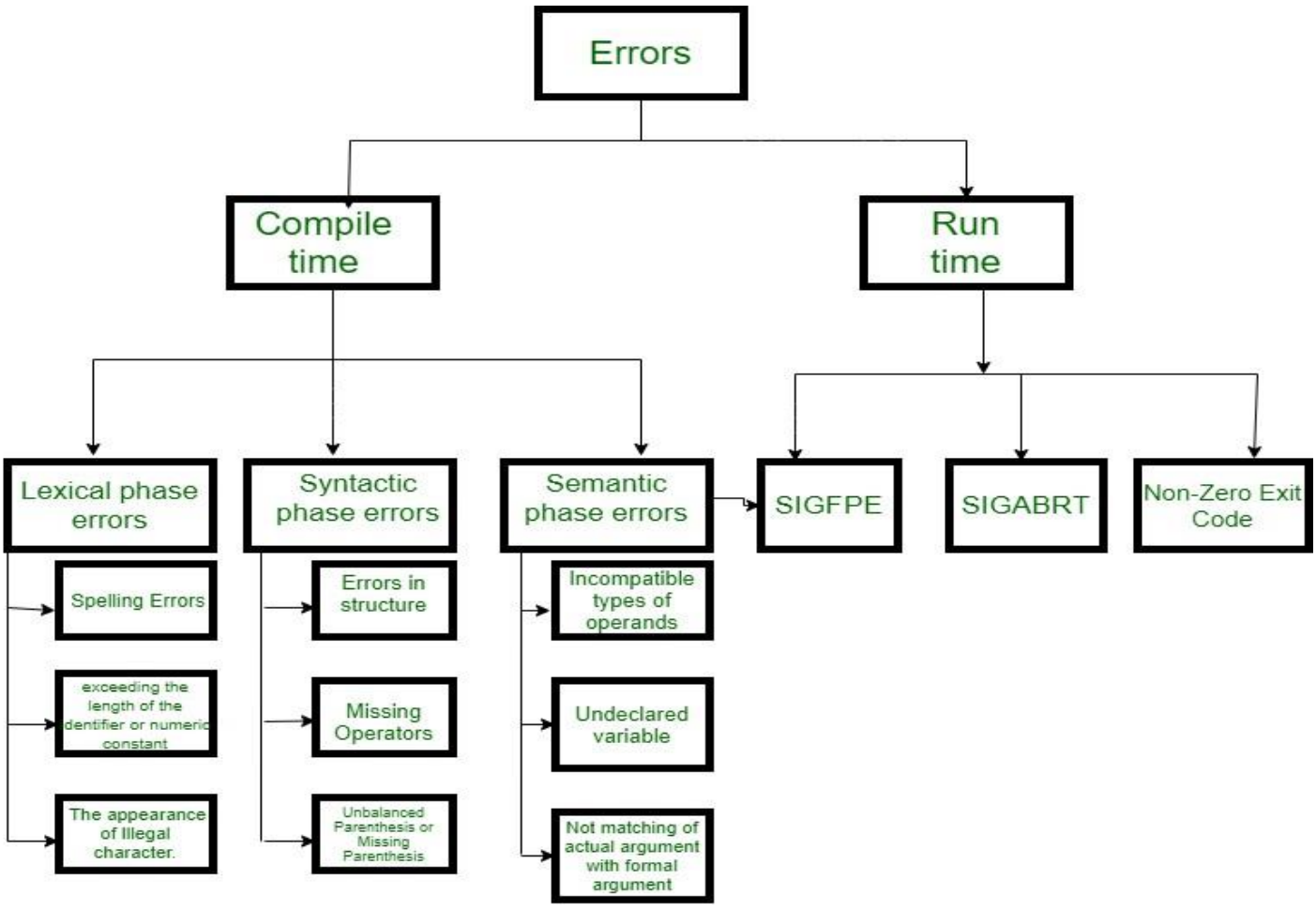
- You need to spend significant time reading the source program and partitioning it in the form of tokens
- Some regular expressions are quite difficult to understand compared to PEG or EBNF rules
- More effort is needed to develop and debug the lexer and its token descriptions
- Additional runtime overhead is required to generate the lexer tables and construct the tokens

LECTURE 6:
ERROR HANDLING

ERROR HANDLING:

A parser should be able to detect and report any error in the program. It is expected that when an error is encountered, the parser should be able to handle it and carry on parsing the rest of the input. Mostly it is expected from the parser to check for errors, but errors may be encountered at various stages of the compilation process.

An **Error** is the blank entries in the symbol table.



Types or Sources of Error –

There are two types of error: run-time and compile-time error:

1. A **run-time error** is an error which takes place during the execution of a program, and usually happens because of adverse system parameters or invalid input data. The lack of sufficient memory to run an application or a memory conflict with another program and logical error are example of this. Logic errors occur when executed code does not produce the expected result. Logic errors are best handled by meticulous program debugging.
2. **Compile-time errors** rises at compile time before execution of the program. Syntax error or missing file reference that prevents the program from successfully compiling is the example of this.

Classification of Compile-time error –

1. **Lexical:** This includes misspellings of identifiers, keywords, or operators
2. **Syntactical**: missing semicolon or unbalanced parenthesis
3. **Semantical**: incompatible value assignment or type mismatches between operator and operand
4. **Logical**: code not reachable, infinite loop.

Semantic Errors

We have mentioned some of the semantics errors that the semantic analyzer is expected to recognize:

- Type mismatch
- Undeclared variable
- Reserved identifier misuse.
- Multiple declaration of variable in a scope.
- Accessing an out of scope variable.
- Actual and formal parameter mismatch.

Finding error or reporting an error – Viable-prefix is the property of a parser which allows early detection of syntax errors.

- **Goal:** detection of an error as soon as possible without further consuming unnecessary input
- **How:** detect an error as soon as the prefix of the input does not match a prefix of any string in the language.

- **Example:** for(**;**), this will report an error as for have two semicolons inside braces.

Error Recovery

The basic requirement for the compiler is to simply stop and issue a message and cease compilation. There are some common recovery methods that are follows.

1. **Panic mode recovery:** This is the easiest way of error-recovery and, it prevents the parser from developing infinite loops while recovering error. The parser discards the input symbol one at a time until one of the designated (like end, semicolon) set of synchronizing tokens (are typically the statement or expression terminators) is found. This is adequate when the presence of multiple errors in same statement is rare. Example: Consider the erroneous expression- (1 + + 2) + 3. Panic-mode recovery: Skip ahead to next integer and then

   continue. Bison: use the special terminal **error** to describe how much input to skip.

   E->int|E+E|(E)|error int|(error)

2. **Phase level recovery:** Perform local correction on the input to repair the error. But error correction is difficult in this strategy.

3. **Error productions:** Some common errors are known to the compiler designers that may occur in the code. Augmented grammars can also be used, as productions that generate erroneous constructs when these errors are encountered. Example: write 5x instead of 5*x
4. **Global correction:** Its aim is to make as few changes as possible while converting an incorrect input string to a valid string. This strategy is costly to implement.